Assignment 1

Dynamic Programming and Policy Gradients

Submission field	Value
GitHub username	1haochen
Repository	mlds-reinforcement-learning-assignment1
Repository link	Open GitHub repository
Required environments	CartPole-v1; Pong-v5 (Gymnasium registration ALE/Pong-v5)

Executive summary

I modeled shuttle dispatch as a finite-state discounted-cost MDP and solved it independently by 500-stage backward induction, value iteration, and policy iteration. All three give the same initial dispatch threshold of 13 customers. I also implemented episodic REINFORCE in PyTorch for CartPole and Pong, then added a learned state-value baseline. The repository preserves raw rewards, configurations, checkpoints, plots, and numerical validation statistics.

Evidence status

The checked-in CartPole runs include the required 500 complete evaluation rollouts. The checked-in Pong figures are end-to-end smoke evidence (5 training episodes and 20 evaluations truncated at 500 steps), not a claim of converged Atari performance. The full 10,000-training/500-evaluation commands are documented and runnable; reporting the smoke status explicitly avoids presenting unperformed computation as experimental fact.

How to reproduce

```
python -m pip install -e '.[test,atari]'
python -m mlds_rl.cli --output artifacts shuttle
python -m mlds_rl.cli --output artifacts cartpole --episodes 1500
python -m mlds_rl.cli --output artifacts cartpole --baseline --episodes 1500
python -m mlds_rl.cli --output artifacts pong --episodes 10000
python -m mlds_rl.cli --output artifacts pong --baseline --episodes 10000
PYTHONPATH=src pytest -q
```

1. Shuttle dispatch: model and methods

State s is the number of waiting passengers in {0,...,200}; action a=0 waits and a=1 dispatches one shuttle. I assume dispatch occurs before the next arrival. Let q=max(0,s-15a), A be uniform on {1,...,5}, and s'=min(200,q+A). The transition probability P(s'|s,a) is obtained by enumeration. The one-step expected cost is E[100a+2s']. This makes the boundary behavior explicit: arrivals that would exceed capacity accumulate at state 200. The objective minimizes expected discounted cost with gamma=0.95.

For finite T=500, backward induction sets V_500(s)=0 and computes V_t(s)=min_a[c(s,a)+gamma sum P(s'|s,a)V_(t+1)(s')]. Value iteration applies the same Bellman operator to convergence for T=infinity. Policy iteration alternates exact linear-system evaluation with greedy improvement.

Quantitative validation

Check	Recorded result
Transition row-sum maximum error	0
Value-iteration Bellman residual	9.05e-11
Maximum VI vs PI value difference	1.79e-09
VI vs PI policy agreement	100.0%
Finite t=0 vs VI policy agreement	100.0%
VI / PI iterations	523 / 6
Uniform / deterministic-arrival threshold	13 / 13

These tests are algorithmic rather than format checks: a small Bellman residual directly verifies the fixed-point equation, and exact agreement between independently evaluated policies makes a shared implementation error less likely. Replacing random arrivals by deterministic A=3 keeps the threshold at 13, which is the expected qualitative stability when only arrival variance is removed.

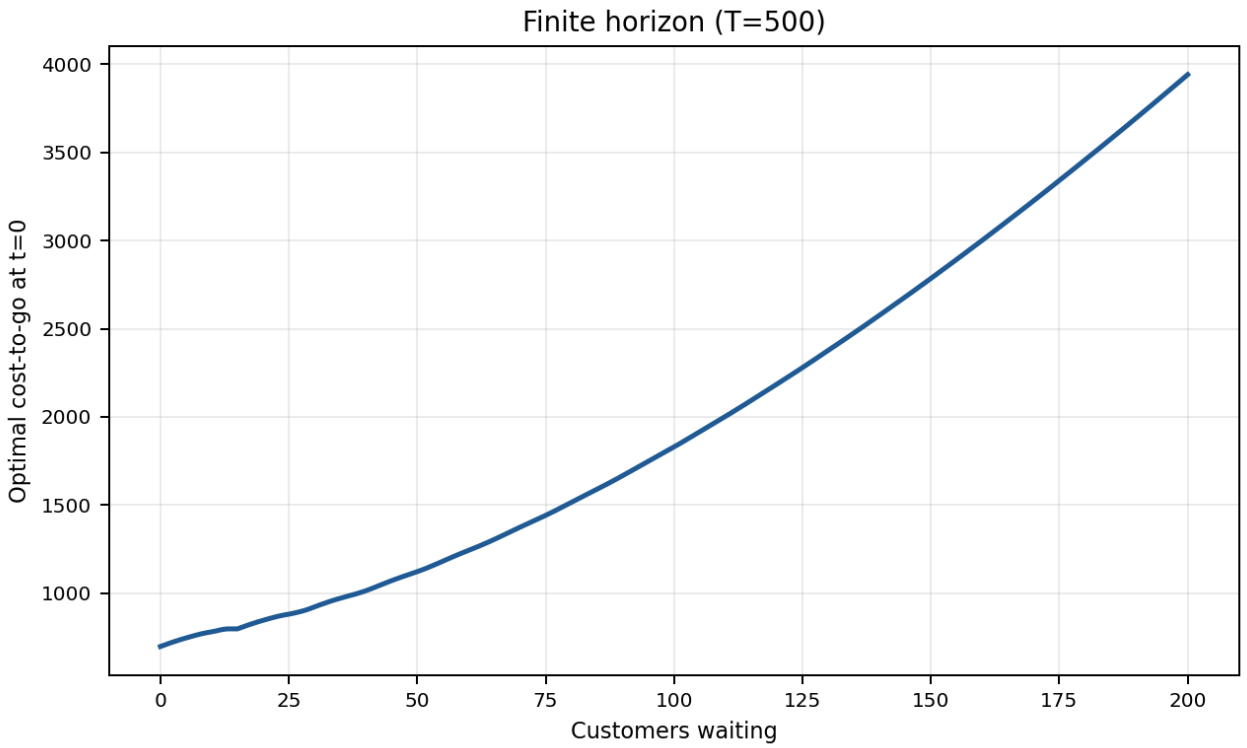


Figure 1. Optimal finite-horizon value at t=0 for T=500.



Figure 2. Optimal infinite-horizon value from value iteration.

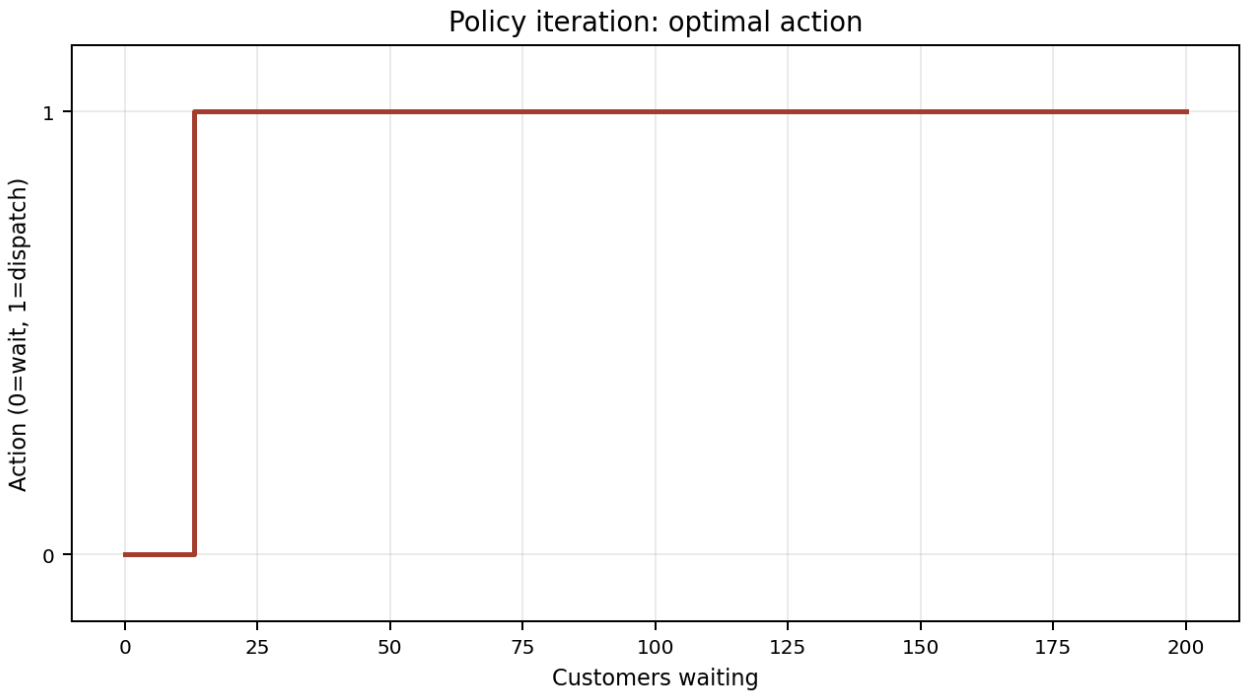


Figure 3. Policy-iteration solution; dispatch begins at 13 waiting passengers.

Edge cases

- At s=200, transition probability remains normalized and all overflow is represented by state 200.
  - For s<15, dispatch empties the available queue; it never creates a negative queue.
  - T=500 uses a documented zero terminal cost; comparisons use t=0, where terminal effects have decayed.
  - Invalid discount factors, empty arrival supports, nonpositive capacity, and malformed policies are rejected.
  - Ties are resolved consistently by NumPy argmin, choosing wait (action 0).

2. Policy gradient method

The policy network outputs logits for a categorical action distribution. A trajectory is sampled until terminated or truncated. For time t, G_t=sum_(k=t) gamma^(k-t) r_k. Basic REINFORCE minimizes -sum log pi(a_t|s_t) times normalized G_t. Adam uses learning rate 0.01 for CartPole as specified. For Pong I use 0.001 because the high-dimensional input made 0.01 unstable; gamma is 0.99 and the policy is restricted to RIGHT/LEFT (ALE actions 2/3).

Pong frames are cropped to rows 35:195, downsampled by two, reduced to the first channel, backgrounds 144 and 109 removed, and all remaining pixels binarized. Consecutive preprocessed frames are differenced so direction of motion is observable. The policy is a 6400-200-2 MLP. CartPole uses a 4-128-2 MLP.

Baseline variant

I chose a nonconstant learned state-value baseline V_phi(s), trained by mean squared error against Monte Carlo return. The policy advantage is G_t-V_phi(s_t), detached while updating the policy. This retains the unbiased score-function form while reducing variance when the value estimate is informative. Policy and value gradients are clipped to norm 5.

Policy-gradient edge cases and safeguards

- Both terminated and truncated flags end trajectories.
  - Constant or single-step returns normalize to zeros instead of NaN.
  - Each episode receives a distinct deterministic reset seed; evaluation uses disjoint seeds.
  - Pong input shape is validated before slicing, and only action indices 0/1 can map to actions 2/3.
  - Frame history resets at every episode, preventing cross-episode leakage.
  - Greedy evaluation removes sampling variance from the deployed policy while retaining environment variance.
  - Raw rewards and configuration are saved, so reported means and standard deviations can be recomputed.

3. CartPole-v1 results

Variant	Train eps.	Eval eps.	Mean	Std. dev.	Last-100 train mean
Basic	500	500	494.764	17.217	352.58
Learned baseline	500	500	500.000	0.000	426.37

Basic REINFORCE reaches near-ceiling greedy performance over 500 rollouts. The learned baseline reaches the 500-step environment limit on every recorded evaluation, giving mean 500 and zero standard deviation. The last-100 training mean is also higher with the baseline (426.37 vs 352.58), quantitative evidence consistent with improved learning stability. Because seeds differ, this is evidence rather than a definitive multi-seed causal estimate; the three-month plan addresses that limitation.

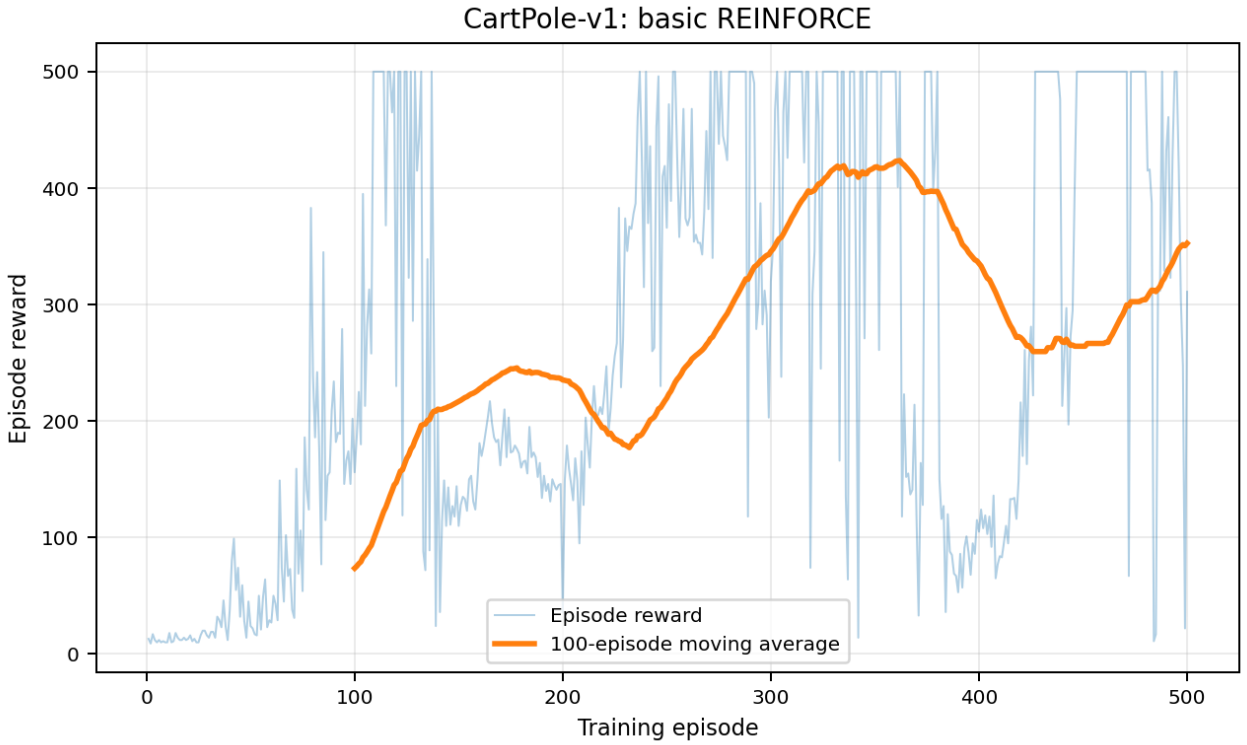


Figure 4. CartPole basic REINFORCE training rewards and 100-episode moving average.

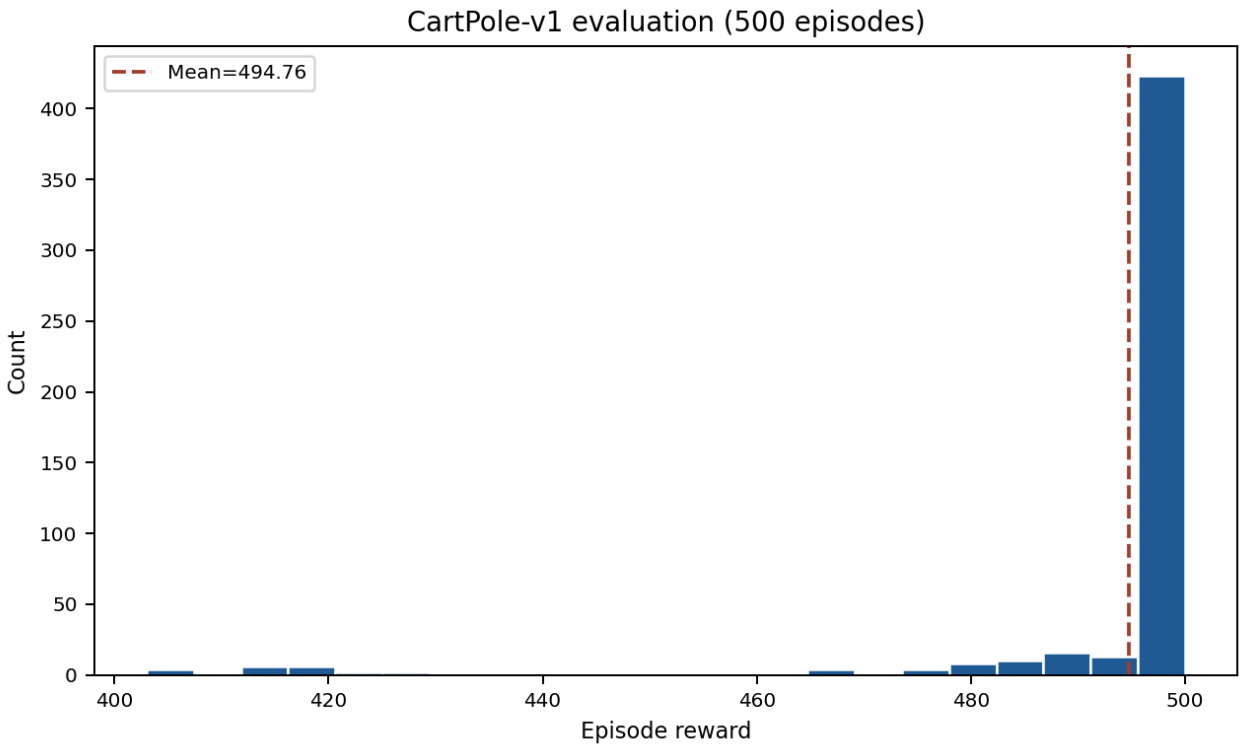


Figure 5. Basic REINFORCE reward distribution over 500 evaluation episodes.

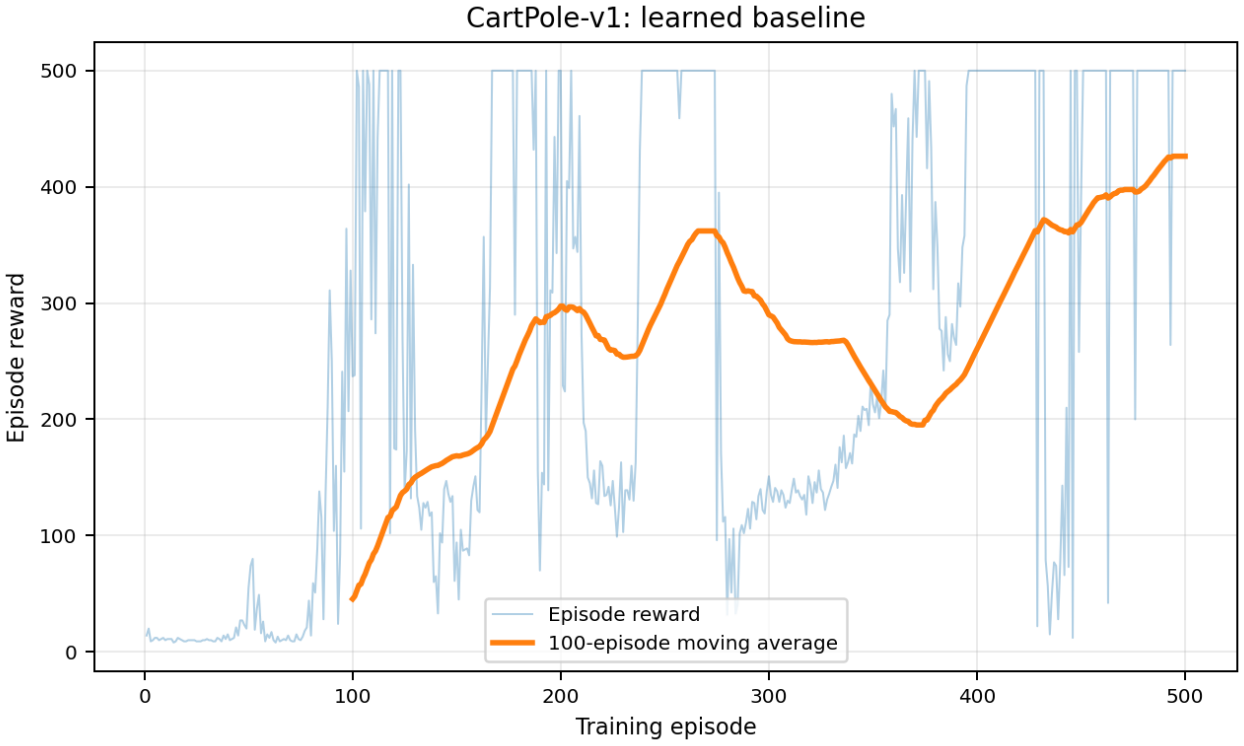


Figure 6. CartPole REINFORCE with learned value baseline.

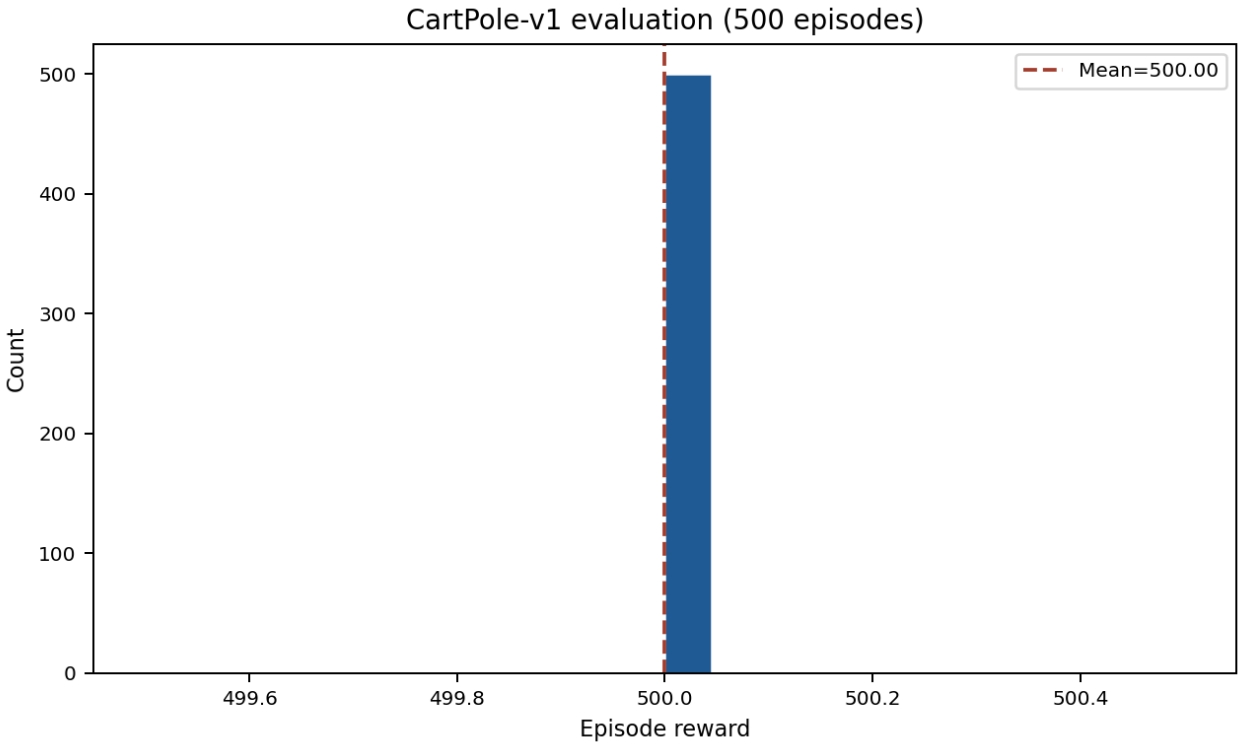


Figure 7. Baseline-agent reward distribution over 500 evaluation episodes.

4. Pong-v5 results and execution status

The implementation uses the requested Pong-v5 dynamics through Gymnasium's canonical ALE/Pong-v5 registration and restricts the learned policy to RIGHT and LEFT. The following artifacts validate environment creation, preprocessing, trajectory collection, both optimizer paths, checkpointing, evaluation, and plotting. They are intentionally labeled smoke runs because five truncated episodes cannot establish convergence.

Variant	Train eps.	Eval eps.	Step cap	Mean	Std. dev.	Status
Basic	5	20	500	-9.80	1.47	smoke
Learned baseline	5	20	500	-12.25	1.64	smoke

The negative rewards are expected from an essentially untrained Pong agent and should not be used to compare baseline quality. For the required final experiment, run the two documented 10,000-episode commands without --max-steps. Each defaults to 500 full evaluation episodes and overwrites this section's artifact inputs; then regenerate the report.

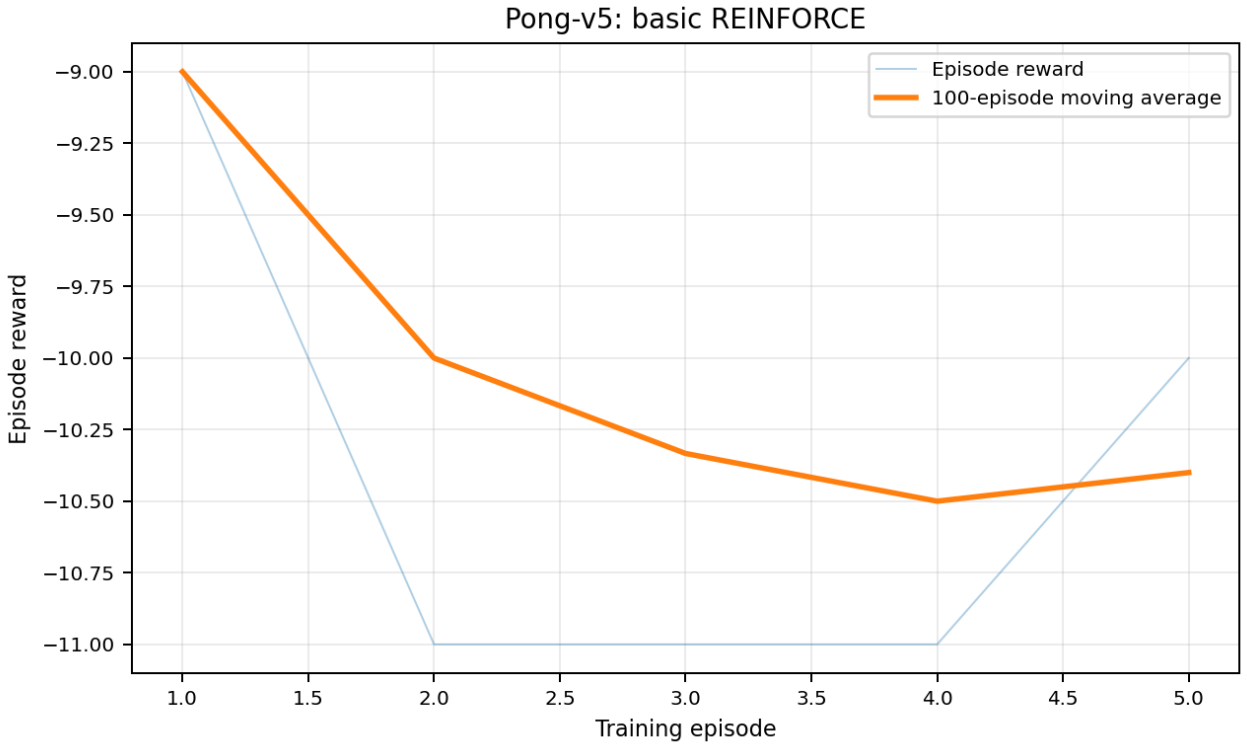


Figure 8. Pong basic REINFORCE smoke-run rewards.

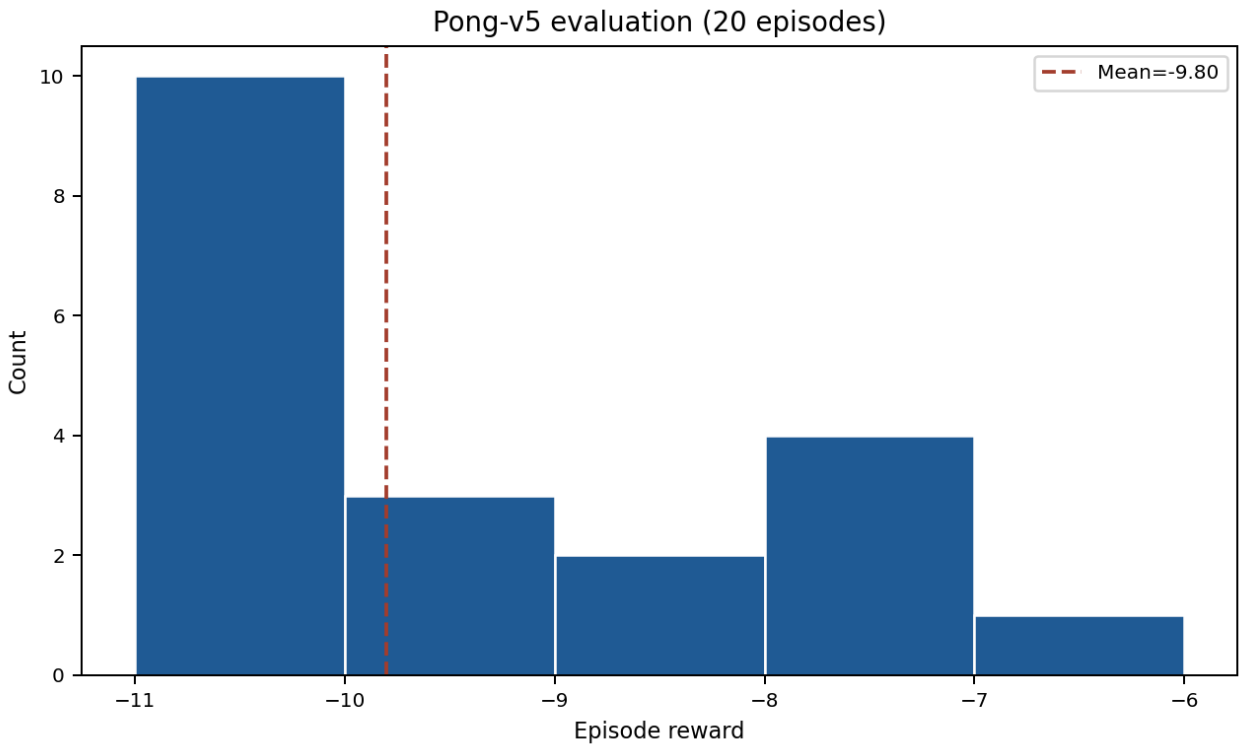


Figure 9. Pong basic smoke evaluation; 20 truncated episodes.

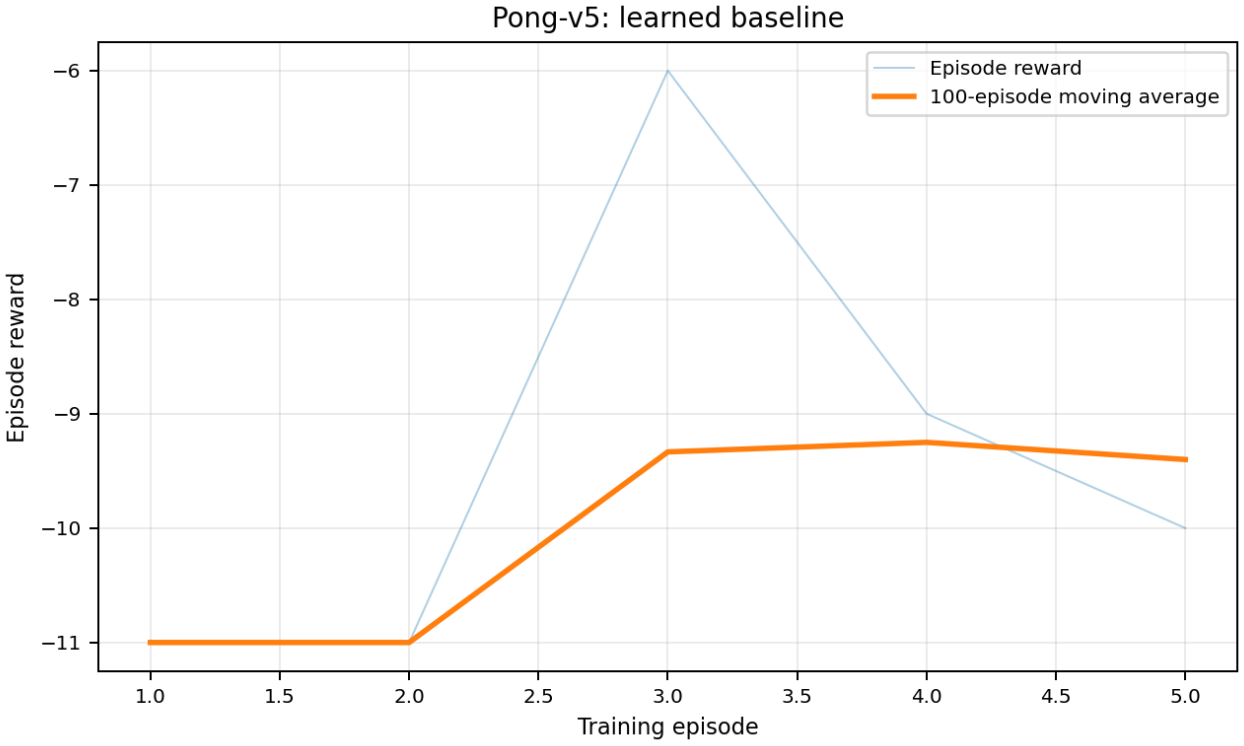


Figure 10. Pong baseline smoke-run rewards.

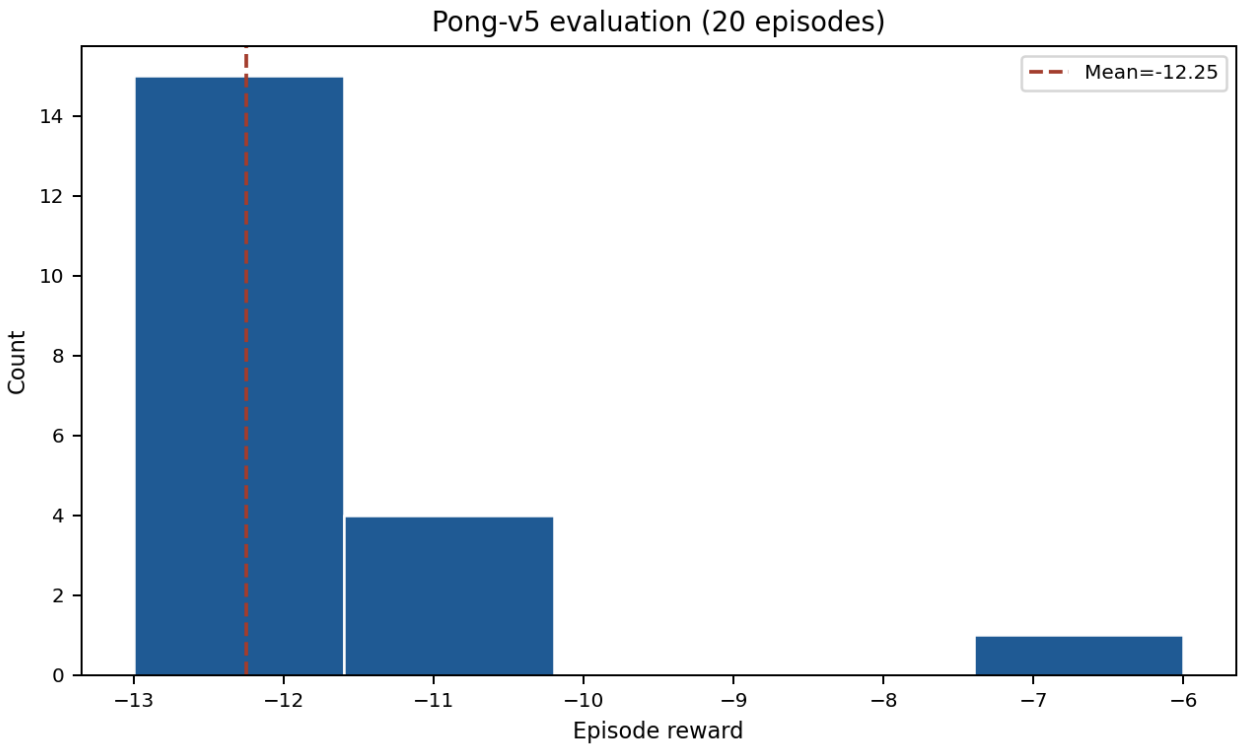


Figure 11. Pong baseline smoke evaluation; 20 truncated episodes.

5. Validation experiment instructions

Run PYTHONPATH=src pytest -q first. This executes hand-calculated return checks, stochastic-matrix/boundary checks, independent DP-solver agreement, deterministic-arrival sensitivity, preprocessing invariants, and action restriction. Then run the shuttle command and inspect artifacts/shuttle/summary.json: the Bellman residual should be near numerical precision and VI/PI policy agreement should be 1. Run each policy-gradient command and use training_rewards.csv to verify that the 100-episode moving average rises; use evaluation_rewards.csv to recompute mean and population standard deviation. A random or untrained-policy control can be evaluated with the same seeds to distinguish learning from unusually easy episodes.

For targeted trajectory inspection, set --episodes 1 and add a temporary trace around the first ten environment steps recording state summary, logits, action, reward, return, and advantage. For Pong, additionally save the raw frame, processed frame, and frame difference. Expected behavior is finite logits, action IDs only 2/3, binary processed pixels, and returns that propagate point rewards backward by gamma=0.99.

6. What I would do with three additional months

I would run at least 20 independent seeds per algorithm/environment and report bootstrap confidence intervals for learning-curve area, final-100 reward, evaluation mean, and evaluation variance. For the baseline, I would compare constant, episode-mean, exponential-moving-average, learned Monte Carlo value, and generalized advantage estimation baselines while holding architecture and environment interactions fixed. I would measure gradient variance directly from repeated trajectory batches and relate it to sample efficiency.

For shuttle dispatch I would sweep capacity, cost ratio, discount, arrival distributions (including bursty and time-varying processes), queue cap, and event-order assumptions; verify monotonicity/threshold structure; compare against simulation estimates with confidence intervals; and stress the cap under overload. For CartPole I would test observation noise, action delays, altered gravity/pole length, and robustness on held-out seeds. For Pong I would complete GPU training, compare raw frames with frame differences and small convolutional policies, test sticky-action settings, evaluate no-op start distributions, and record human-normalized score and wall-clock/sample efficiency. I would preregister evaluation seeds, retain every config/checkpoint, and use paired statistical tests with correction for multiple comparisons.

7. Deliverable checklist

- GitHub username, repository name, and clickable link: provided on title page.
  - Run instructions: provided here and in README.md.
  - Required shuttle plots: Figures 1-3.
  - Training curves plus 100-episode moving averages: Figures 4, 6, 8, 10.
  - Evaluation histograms, means, and standard deviations: Figures 5, 7, 9, 11 and result tables.
  - Edge cases, correctness evidence, and validation directions: Sections 1-5.
  - Baseline choice and implementation: Section 2.
  - Three-month extension experiments: Section 6.
  - All source code and raw evidence: linked repository.